왜 4-1if 조건문과 4-2가 [4] 결과 출력 다음에 있는거야? [4] 전에 있어야 하는 것 아니야?

코드의 구조를 보면 그렇게 생각하실 수 있습니다! 보통 [계산]을 다 끝내고 [출력]을 하는 것이 일반적인 흐름이기 때문이죠.

하지만 현재 코드에서 `if` 조건문들이 **[4] 결과 출력** 영역 안에 포함되어 있는 데에는 두 가지 공학적인 이유가 있습니다.

---

1. "판단"과 "출력"이 동시에 일어나는 구조

현재 프로그램에서 `if` 문은 단순히 값을 계산하는 것이 아니라, **어떤 문장을 화면에 보여줄지 결정**하는 역할을 합니다.

- **4-1 체급 분류**: "남성 페더급"이라고 출력할지, "여성 라이트급"이라고 출력할지 결정함과 동시에 바로 `printf` 를 실행합니다. 만약 이를 **[4]** 이전에 두려면 따로 문자열 변수를 만들어 저장해야 하는데, 이는 아직 배우지 않은 복잡한 개념(배열/포인터)이 필요합니다.
- **4-2 승급 판별**: 모든 데이터 분석(퍼포먼스 지수 등)이 끝난 후, 최종 리포트의 **마지막 문구**로서 결과를 보여주기 위해 출력부 제일 하단에 배치한 것입니다.

2. 논리적 흐름 (Data -> Information)

공학적으로 프로그램은 **데이터 수집** → **가공(연산)** → **정보 출력**의 단계를 거칩니다.

- **[3] 산술 연산**: `performance_idx` 라는 핵심 수치를 먼저 계산합니다.
- **[4] 결과 출력**: 계산된 수치(`performance_idx`)와 기존 데이터(`weight` , `months`)를 사용하여, 사용자가 읽기 좋은 형태(Report)로 화면에 뿌려주는 단계입니다. 이때 `if` 문을 사용하여 사용자 맞춤형 메시지를 실시간으로 골라 출력하는 것이 현재 단계에서 가장 효율적인 방식입니다.

💡 요약하자면

만약 `if` 문 안에서 새로운 **값**을 계산하고 그 값을 나중에 사용해야 한다면 **[4]** 앞에 오는 것이 맞습니다. 하지만 지금은 "조건에 맞는 문장을 화면에 보여주는 것"이 목적이기 때문에, 리포트를 출력하는 **[4]** 영역 안에서 분기 처리를 하는 것이 논리적으로 자연스럽습니다.

🥋 성별에 따른 다중/중첩 조건문 설계

여성 관원의 체급 기준을 반영하기 위해, 1차 과제에는 없었던 `user_gender` 변수를 추가하고 이를 **중첩 if문**의 첫 번째 관문으로 설정합니다.

1. 변수 추가 (char 자료형)

- `char user_gender;` : 사용자가 남성(M)인지 여성(F)인지 입력받습니다.

2. 중첩 조건문을 활용한 체급 판별 (Nested IF)

가장 바깥쪽 `if` 에서 성별을 먼저 나누고, 그 안에서 다시 `else if` 를 통해 세부 체급을 나누는 방식입니다.

- **남성용 로직 (M):**
  - 64kg 미만: 페더급
  - 70kg 미만: 라이트급
  - 70kg 이상: 미들급 이상
- **여성용 로직 (F):**
  - 53kg 미만: 페더급
  - 58kg 미만: 라이트급
  - 58kg 이상: 미들급 이상

여성 관원들도 있기 때문에 사용자의 체급을 좀 더 많이 나눠야할 것 같아.

✦

주짓수는 성별에 따라 체급 체계가 다르므로, 성별(Gender)이라는 새로운 변수를 추가하여 조건문을 더 정교하게 발전시키면 훨씬 완성도 높은 프로그램이 됩니다.

2차 과제의 핵심인 **다중 조건문**과 **중첩 조건문**을 활용하여 남성과 여성 관원의 체급을 분리해 판별하는 로직을 상세히 설명해 드릴게요.

🥋 성별에 따른 다중/중첩 조건문 설계

여성 관원의 체급 기준을 반영하기 위해, 1차 과제에는 없었던 `user_gender` 변수를 추가하고 이를 **중첩 if문**의 첫 번째 관문으로 설정합니다.

1. 변수 추가 (char 자료형)

- `char user_gender;` : 사용자가 남성(M)인지 여성(F)인지 입력받습니다.

2. 핵심 로직 설계 (다중/중첩 조건문 활용)

① 다중 조건문 (`if ~ else if ~ else`): 체급 자동 분류

입력받은 `current_weight` (체중)를 기준으로 주짓수 대회 규정에 따른 체급을 자동으로 알려줍니다.

- **예시**: 64kg 미만은 '페더급', 70kg 미만은 '라이트급', 그 이상은 '미들급'으로 분류.

② 논리 연산자 (`&&`): 승급 심사 자격 판별

두 가지 이상의 조건을 동시에 만족해야 하는 복합 상황을 판별합니다.

- **조건**: 수련 기간이 **6개월 이상**이면서(`&&`) 성장 지수가 **50점 이상**인 경우에만 "승급 심사 대상자"로 분류.

③ 중첩 조건문: 등급별 맞춤 조언

조건문 안에 또 다른 조건문을 넣어 정교한 피드백을 줍니다.

- **로직**: 먼저 '에이스(80점 이상)' 등급인지 확인하고, 그 안에서 다시 '스파링 횟수'를 체크하여 "기술은 좋으나 체력 보충이 필요함" 등의 디테일한 조언 출력.

1차 과제에 이어서 2차 과제를 통해 프로그램을 발전시키려고해.
2차 과제물의 과제목표와 주요내용 : 조건문을 활용한 데이터 분기 처리
핵심 프로그래밍 기술 및 개념 : 다중 조건문 if ~else if else / 중첩 조건문 : if 문 안의 if문 / 논리/ 관계 연산자 :&&(AND).

1차 과제(V1.0)에서 데이터를 단순히 **입력하고 계산**하는 것에 집중했다면, 2차 과제(V2.0)는 입력된 데이터를 바탕으로 프로그램이 판단(Decision Making)을 내리게 만드는 단계입니다.

제시해주신 과제 목표에 맞춰 **주짓수 퍼포먼스 분석 시스템**을 어떻게 발전시킬지 주요 시나리오와 핵심 로직을 정리해 드립니다.

🥋 주짓수 시스템 V2.0: 조건문을 활용한 지능형 진단

1. 과제 목표 및 주요 내용

- **목표**: 1차에서 계산된 '성장 지수'와 '체중' 데이터를 활용하여 **체급 분류, 등급 산정, 승급 가능성 판별** 등의 분기 처리를 구현합니다.
- **주요 내용**: 사용자의 상태를 분석하여 단순 수치가 아닌 **텍스트 기반의 피드백**을 제공하는 시스템으로 발전시킵니다.